



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SC1007: DATA STRUCTURES AND ALGORITHMS

Introduction & memory management in Python

Introduction to Memory Management

- Memory management refers to the critical process of allocating, deallocating, and coordinating computer memory effectively.
- The primary objective of memory management is to ensure that all processes execute smoothly and efficiently utilize system resources.
- In Python, this is achieved through a combination of **dynamic memory allocation**, **automatic garbage collection**, and **reference counting mechanisms**.

Programs need memory to run
Memory is a limited resource

Memory Management Features in Python

- **Automatic Memory Management:** Python handles memory allocation/deallocation
- **Memory Pooling:** Reuse of previously allocated memory blocks
- **Memory Fragmentation Prevention:** Efficient organization of memory blocks
- Benefits: -
 - Prevents memory leaks
 - Optimizes memory usage
 - Reduces development complexity
 - Improves application performance

Poor memory management leads to crashes and slow programs

Understanding Python Memory Management: Names, References, and Objects

- **Names:** In Python, variables are essentially **naming that reference** or **point to objects** in memory. When you create a variable, it doesn't directly hold the **object itself** but acts as a **label** or **identifier**.
- **References:** A reference is the link between the name and the object. This reference allows Python to manage and access the object in memory without directly storing it in the variable.
- **Objects:** Objects are the actual entities stored in memory. These can be data types such as integers, strings, lists, or user-defined objects. Every object has its own memory address, and Python keeps track of them using references.

Understanding Python Memory Management: Names, References, and Objects

```
x = 42
```

```
print(id(x)) # e.g., 140712834927920
```

Name

Reference

Object

id(x) = 140712834927920

Name (Variable):

x

Reference:

140712834927920

Object:

42



Python Memory Layout: Code, Stack, and Heap Segments

CODE SEGMENT

- Python Bytecode
- Program Instructions

```
pi = 3.14159 # Global variable  
x = 42         list1 = [1, 2, 3]
```

STATIC/GLOBAL SEGMENT

- Global Variables
- Static Data

pi → 0x9999

STACK

- Function Call Stack
- Local Variables and References

Local Variables:

x
0x1234

list1
0x5678

main() Frame

function() Frame

HEAP (Dynamic Memory)

- Objects and Instance Variables
- Lists, Dictionaries, User-defined Objects

42

[1, 2, 3]

3.14159

Code Segment: This segment contains the compiled Python bytecode and program instructions.

Python Memory Layout: Code, Stack, and Heap Segments

CODE SEGMENT

- Python Bytecode
- Program Instructions

```
pi = 3.14159 # Global variable  
x = 42         list1 = [1, 2, 3]
```

STATIC/GLOBAL SEGMENT

- Global Variables
- Static Data

pi → 0x9999

STACK

- Function Call Stack
- Local Variables and References

Local Variables:

x → 0x1234
list1 → 0x5678

main() Frame
function() Frame

HEAP (Dynamic Memory)

- Objects and Instance Variables
- Lists, Dictionaries, User-defined Objects

42 [1, 2, 3] 3.14159

Code Segment: This segment contains the compiled Python bytecode and program instructions.

Static/Global Segment: This segment stores global variables and static data that are available throughout the program's execution.

Python Memory Layout: Code, Stack, and Heap Segments

CODE SEGMENT

- Python Bytecode
- Program Instructions

```
pi = 3.14159 # Global variable  
x = 42         list1 = [1, 2, 3]
```

STATIC/GLOBAL SEGMENT

- Global Variables
- Static Data

pi → 0x9999

STACK

- Function Call Stack
- Local Variables and References

Local Variables:

x
0x1234

list1
0x5678

main() Frame

function() Frame

HEAP (Dynamic Memory)

- Objects and Instance Variables
- Lists, Dictionaries, User-defined Objects

42

[1, 2, 3]

3.14159

Code Segment: This segment contains the compiled Python bytecode and program instructions.

Static/Global Segment: This segment stores global variables and static data that are available throughout the program's execution.

Stack: The stack is used for function call frames, local variables, and references.

Python Memory Layout: Code, Stack, and Heap Segments

CODE SEGMENT

- Python Bytecode
- Program Instructions

```
pi = 3.14159 # Global variable  
x = 42        list1 = [1, 2, 3]
```

STATIC/GLOBAL SEGMENT

- Global Variables
- Static Data

pi → 0x9999

STACK

- Function Call Stack
- Local Variables and References

Local Variables:

x
0x1234

list1
0x5678

main() Frame

function() Frame

HEAP (Dynamic Memory)

- Objects and Instance Variables
- Lists, Dictionaries, User-defined Objects

42

[1, 2, 3]

3.14159

Code Segment: This segment contains the compiled Python bytecode and program instructions.

Static/Global Segment: This segment stores global variables and static data that are available throughout the program's execution.

Stack: The stack is used for function call frames, local variables, and references.

Heap: The heap is used to store dynamically allocated objects, such as lists, dictionaries, or user-defined objects. Managed dynamically by Python's garbage collector.

Memory Characteristics of Immutable Objects (Integers)

```
x = 42
y = x
```

Before: x = 42, y = x

CODE SEGMENT

- Python Bytecode x = 42
- Program Instructions y = x

STACK

- Local Variables and References

x, y (references) → 0x1234

HEAP (Dynamic Memory)

- Objects and Instance Variables

Integer Object: 42

```
x = 42
y = x
x += 1 # x becomes 43, but y stays 42
```

After: x += 1

CODE SEGMENT

- Python Bytecode x += 1
- Program Instructions # x becomes 43, y stays 42

STACK

- Local Variables and References

x → 0x5678

y → 0x1234

HEAP (Dynamic Memory)

- Objects and Instance Variables

Integer: 43

Integer: 42

Memory Characteristics of Immutable Objects (Strings)

```
s1 = "hello"  
s2 = "hello" # Same object reused  
s1 = "hello!" # # Creates new object, s2 keeps old reference
```

Before: Initial Assignment

CODE SEGMENT

```
s1 = "hello"  
s2 = "hello"
```

STACK

s1, s2 → 0x7f8a

HEAP

"hello" (0x7f8a)
refcount: 2

After: s1 Reassignment

CODE SEGMENT

```
s1 = "hello!"  
# s2 still "hello"
```

STACK

s1 → 0x7f9b

s2 → 0x7f8a

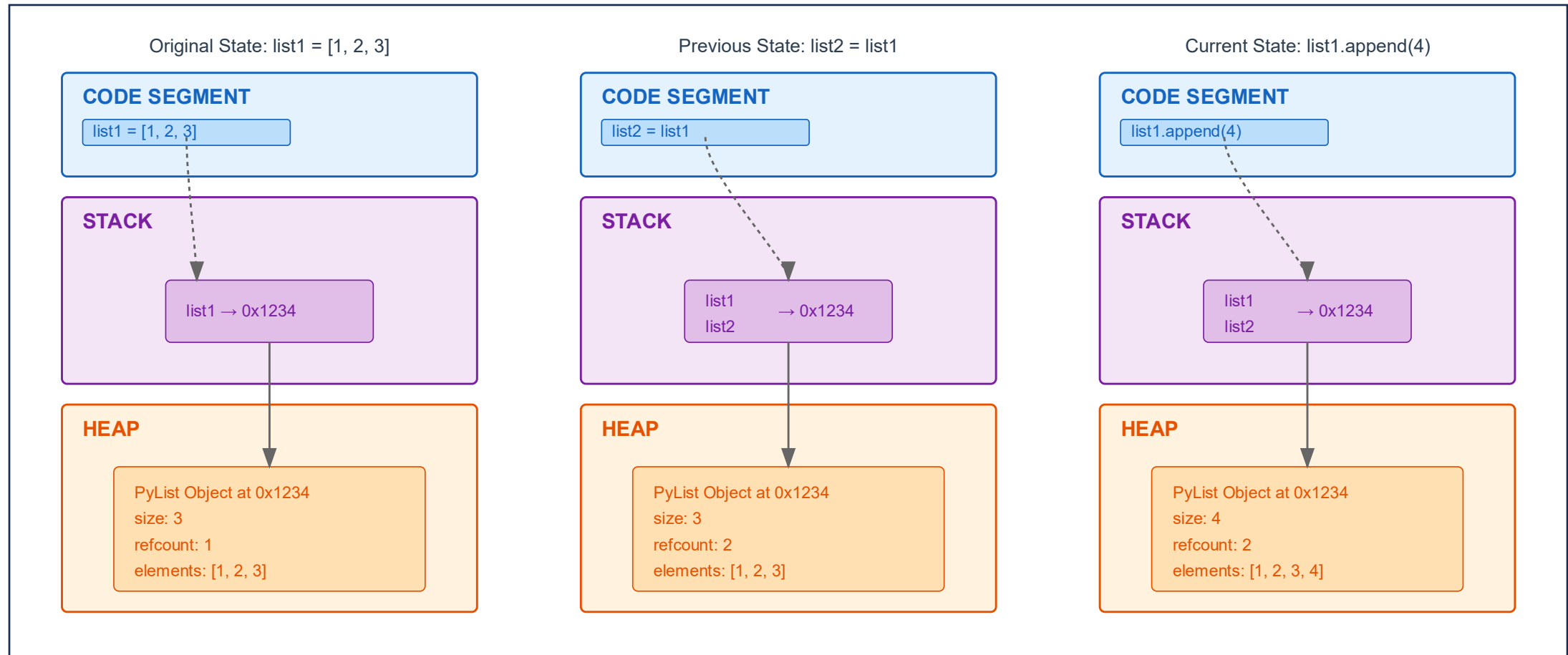
HEAP

"hello!"
refcount: 1

"hello"
refcount: 1

Mutable Objects (list, dict, set)Memory Characteristics

```
list1 = [1, 2, 3] # Create list, list1 points to it
list2 = list1 # list2 points to same list
list1.append(4) # Modify the list object
```



Introduction to Memory Management

- Memory management in Python is handled automatically
- Multiple mechanisms work together to manage memory efficiently
- Understanding these concepts helps write better Python code
- Core Memory Management Mechanisms
 - Reference Counting
 - Garbage Collection
 - Memory Pooling
 - Memory Interning
 - Manual Memory Management

1. Reference Counting

Reference Counting

- Primary memory management mechanism in Python
- Each object maintains a count of references pointing to it
- When count reaches zero, object is automatically deallocated
- Immediate memory reclamation
- Handles most common memory management scenarios

Reference Counting: Advantages

1. Predictable Performance

- Immediate cleanup of unused objects
- No unexpected pauses in execution
- Memory freed as soon as it's no longer needed

Reference Counting: Advantages

1. Predictable Performance

- Immediate cleanup of unused objects
- No unexpected pauses in execution
- Memory freed as soon as it's no longer needed

2. Deterministic Resource Management

- Resources released promptly
- Efficient for memory-intensive applications
- Good for real-time systems

Reference Counting: Advantages

1. Predictable Performance

- Immediate cleanup of unused objects
- No unexpected pauses in execution
- Memory freed as soon as it's no longer needed

2. Deterministic Resource Management

- Resources released promptly
- Efficient for memory-intensive applications
- Good for real-time systems

3. Simple Implementation

- Easy to understand and maintain
- Transparent to developers
- Automatic memory management

Reference Counting: Advantages

1. Predictable Performance

- Immediate cleanup of unused objects
- No unexpected pauses in execution
- Memory freed as soon as it's no longer needed

2. Deterministic Resource Management

- Resources released promptly
- Efficient for memory-intensive applications
- Good for real-time systems

3. Simple Implementation

- Easy to understand and maintain
- Transparent to developers
- Automatic memory management

4. Memory Efficiency

- Reduced memory footprint
- Better cache utilization

Python Reference Counting Example

- When a list, such as `[1, 2, 3]`, is assigned to a variable (e.g., `x`), its reference count is initially set to 1.
- If additional references are created, such as through the assignments `y = x` and `z = x`, the reference count increases to 3.
- Conversely, as references are removed using the `del` statement, the reference count decreases.
- Once the reference count reaches zero, Python's memory management system automatically deallocates the memory associated with the list. This mechanism ensures efficient memory usage and prevents memory leaks in applications.

Python Reference Counting Example

```
1  # Import required modules
2  import sys      # For checking reference counts
3
4  # Create list and first reference
5  x = [1, 2, 3]
6  print(sys.getrefcount(x) - 1)
7
8  # Add second reference
9  y = x
10 print(sys.getrefcount(y) - 1)
11
12 # Add third reference
13 z = x
14 print(sys.getrefcount(z) - 1)
15
16 # Remove first reference
17 del x
18
19 # Remove second reference
20 del y
21
22 # Remove final reference
23 del z
24
25 # Object now eligible for garbage collection
26 # Force garbage collection (optional)
```

Python Reference Counting Example

```
1 # Import required modules
2 import sys      # For checking reference counts
3
4 # Create a list and assign it to variable x
5 x = [1, 2, 3]  # Reference count = 1
6 # Print current reference count minus 1
7 # (subtract 1 because getrefcount() creates a temporary reference)
8 print(sys.getrefcount(x) - 1) # Output: 1
9 # Memory state: One reference (x) pointing to [1, 2, 3]
```

Step 1: x = [1, 2, 3]

- Python Bytecode x = [1, 2, 3]
- Program Instructions

- Local Variables and References

x → 0x1234

- Objects and Instance Variables

[1, 2, 3]
refcount: 1

Python Reference Counting Example

```
1  # Import required modules
2  import sys      # For checking reference counts
3
4  # Create a list and assign it to variable x
5  x = [1, 2, 3]   # Reference count = 1
6  # Print current reference count minus 1
7  # (subtract 1 because getrefcount() creates a temporary reference)
8  print(sys.getrefcount(x) - 1) # Output: 1
9  # Memory state: One reference (x) pointing to [1, 2, 3]
10
11 # Create another reference y pointing to the same list object
12 y = x # Reference count = 2
13 print(sys.getrefcount(y) - 1) # Output: 2
14 # Memory state: Two references (x, y) pointing to [1, 2, 3]
```

Step 2: $y = x$

- Python Bytecode $y = x$
- Program Instructions

- Local Variables and References

x, y (references) → 0x1234

- Objects and Instance Variables

[1, 2, 3]
refcount: 2

Python Reference Counting Example

```
1  # Import required modules
2  import sys      # For checking reference counts
3
4  # Create a list and assign it to variable x
5  x = [1, 2, 3]   # Reference count = 1
6  # Print current reference count minus 1
7  # (subtract 1 because getrefcount() creates a temporary reference)
8  print(sys.getrefcount(x) - 1) # Output: 1
9  #Memory state: One reference (x) pointing to [1, 2, 3]
10
11 # Create another reference y pointing to the same list object
12 z = x           # Reference count = 2
13 print(sys.getrefcount(y) - 1) # Output: 2
14 # Memory state: Two references (x, y) pointing to [1, 2, 3]
15
16 # Create third reference z pointing to the same list object
17 z = x           # Reference count = 3
18 print(sys.getrefcount(z) - 1) # Output: 3
19 # Memory state: Three references (x, y, z) pointing to [1, 2, 3]
```

Step 3: z = x

- Python Bytecode z = x
- Program Instructions

- Local Variables and References

x, y, z (references) → 0x1234

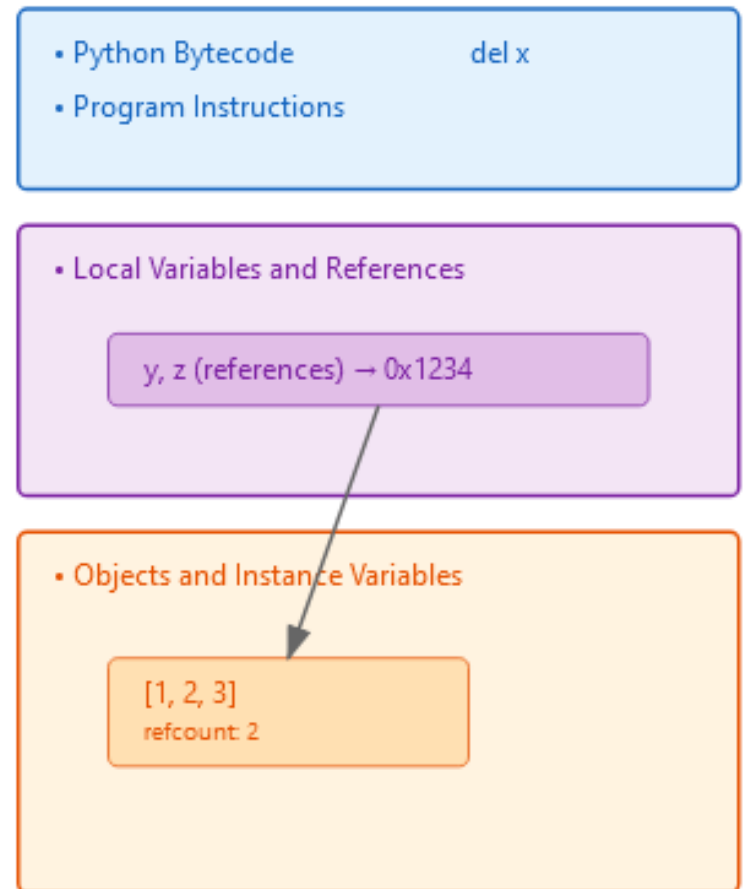
- Objects and Instance Variables

[1, 2, 3]
refcount: 3

Python Reference Counting Example

```
20 # Delete x reference
21 del x # Reference count = 2
22 # Memory state: refcount decreases to 2
23 # Two references remain (y, z) pointing to [1, 2, 3]
```

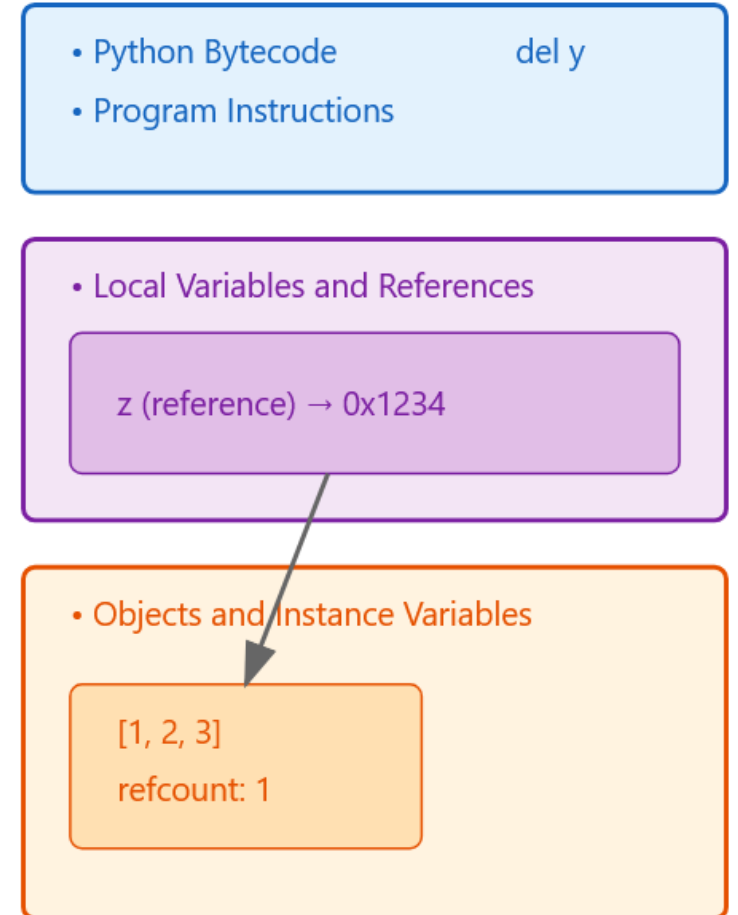
After: del x



Python Reference Counting Example

```
20 # Delete x reference
21 del x # Reference count = 2
22 # Memory state: refcount decreases to 2
23 # Two references remain (y, z) pointing to [1, 2, 3]
24
25 # Delete y reference
26 del y # Reference count = 1
27 # Memory state: refcount decreases to 1
28 # One reference remains (z) pointing to [1, 2, 3]
```

After: del y



Python Reference Counting Example

```
20 # Delete x reference
21 del x # Reference count = 2
22 # Memory state: refcount decreases to 2
23 # Two references remain (y, z) pointing to [1, 2, 3]
24
25 # Delete y reference
26 del y # Reference count = 1
27 # Memory state: refcount decreases to 1
28 # One reference remains (z) pointing to [1, 2, 3]
29
30 # Delete z reference
31 del z # Reference count = 0
32 # Memory state: refcount becomes 0
33 # No references remain, object becomes eligible for garbage collection
34 # Object is marked as eligible for garbage collection
```

After: del z

- Python Bytecode del z
- Program Instructions

- Local Variables and References

No references remain

- Objects and Instance Variables

[1, 2, 3]

refcount: 0

Eligible for GC

Python Reference Counting Example

```
1  # Import required modules
2  import sys                # For checking reference counts
3
4  # Create list and first reference
5  x = [1, 2, 3]             # Creates list object in heap # Initial reference count = 1
6  print(sys.getrefcount(x) - 1)    # Output: 1
7
8  # Add second reference
9  y = x                     # y points to same object as x # Increases reference count to 2
10 print(sys.getrefcount(x) - 1)    # Output: 2 # Shows two references (x and y)
11
12 # Add third reference
13 z = x                     # z points to same object as x and y # Increases reference count to 3
14 print(sys.getrefcount(x) - 1)    # Output: 3 # Shows three references (x, y, and z)
15
16 # Remove first reference
17 del x                     # Removes x's reference # Decreases count to 2 # Object still exists in memory
18
19 # Remove second reference
20 del y                     # Removes y's reference # Decreases count to 1 # Object still exists in memory
21
22 # Remove final reference
23 del z                     # Removes final reference # Reference count becomes 0
24
25 # Object now eligible for garbage collection
26 # Force garbage collection (optional)
27 # Not typically needed # Python automatically handles cleanup # Returns number of objects collected
```

Reference Counting: Disadvantages

1. Overhead Costs

- Additional memory for reference counts
- CPU cycles needed for updating counters

Reference Counting: Disadvantages

1. Overhead Costs

- Additional memory for reference counts
- CPU cycles needed for updating counters

2. Circular References

- Cannot handle circular references alone
- Requires additional garbage collector
- Potential memory leaks if not managed

Reference Counting: Disadvantages

1. Overhead Costs

- Additional memory for reference counts
- CPU cycles needed for updating counters

2. Circular References

- Cannot handle circular references alone
- Requires additional garbage collector
- Potential memory leaks if not managed

3. Threading Complication

- Reference counting must be thread-safe
- Requires atomic operations
- Can impact performance in multi-threaded apps

Reference Counting: Circular References

Reference Counting with Circular References occurs when two or more objects or data structures reference each other in a cycle. The problem arises because each object maintains a non-zero reference count due to the circular dependency, even when they become unreachable from the rest of the program.

- Lists referencing each other create cycles
- Memory stays allocated due to internal references
- Reference count never reaches zero
- Garbage collection needed to detect and free circular references

Reference Counting: Circular References

```
1  # Create two lists
```

```
2  list1 = []
```

```
3  list2 = []
```

```
5  # Create circular reference
```

```
6  list1.append(list2) # list2 referenced by list1
```

```
7  list2.append(list1) # list1 referenced by list2
```

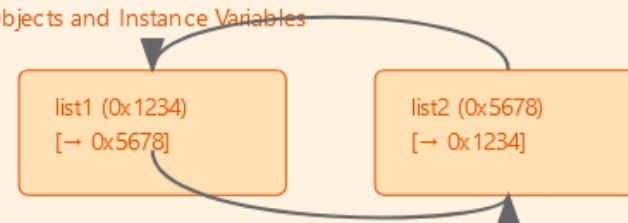
Before Deletion: Circular Reference

- Python Bytecode `list1.append(list2)`
- Program Instructions `list2.append(list1)`

- Local Variables and References

list1 → 0x1234
list2 → 0x5678

- Objects and Instance Variables



Reference Counting: Circular References

```
1  # Create two lists
```

```
2  list1 = []
```

```
3  list2 = []
```

```
4  
5  # Create circular reference
```

```
6  list1.append(list2) # list2 referenced by list1
```

```
7  list2.append(list1) # list1 referenced by list2
```

```
8  
9  # Even after deletion, lists still reference each other
```

```
10 del list1 # Still referenced by list2[0]
```

```
11 del list2 # Still referenced by list1[0]
```

```
12  
13 # Garbage collector detects these objects are unreachable
```

```
14 # Memory will be freed in next GC cycle
```

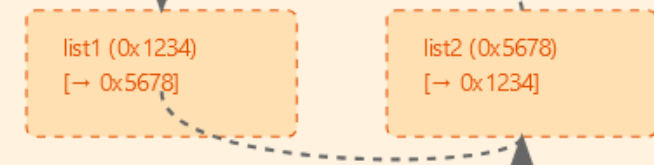
After Deletion: Unreachable Cycle

- Python Bytecode `del list1`
- Program Instructions `del list2`

- Local Variables and References

No references in scope

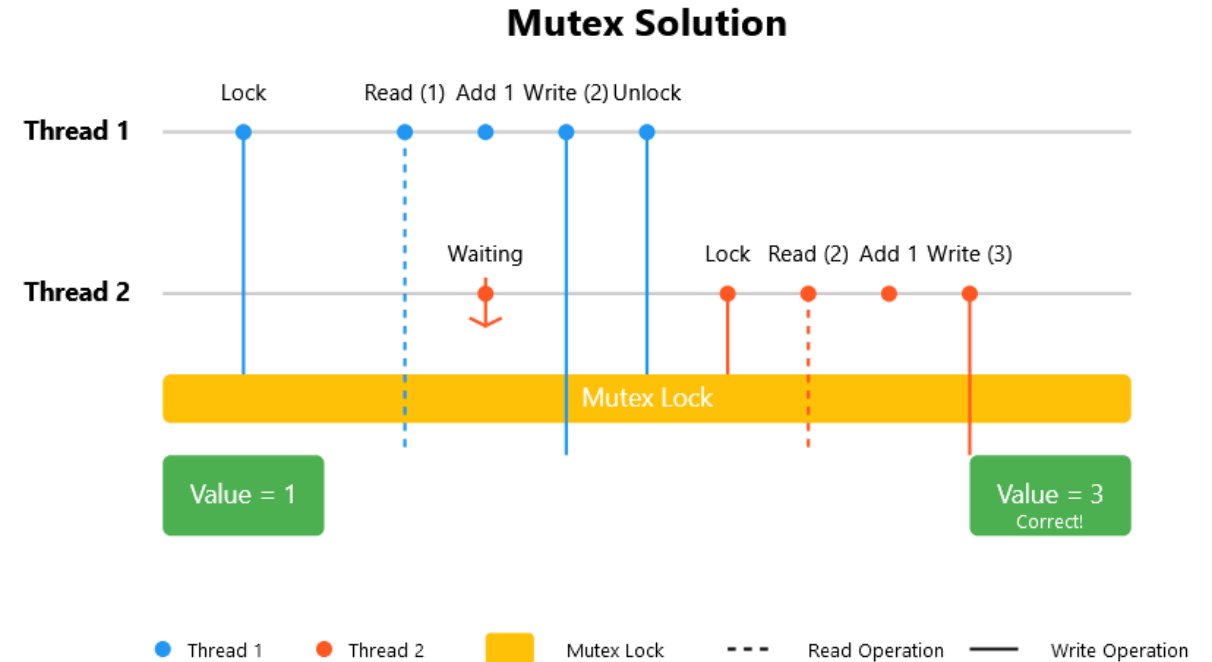
- Objects and Instance Variables



Cycle detected by GC - Both objects will be collected

Reference Counting: Mutex Solution

- The Mutex Solution diagram shows how to prevent race conditions using mutual exclusion locks.
- Thread 1 (**blue**) first acquires the lock, reads value 1, adds 1, writes 2, then releases the lock. During this time, Thread 2 (**red**) must wait.
- Only after Thread 1 releases the lock can Thread 2 acquire it, read value 2, add 1, and write 3.



2. Garbage Collection

Garbage Collection

- **Automatic Memory Management:** Detects and removes unused objects to optimize memory usage, eliminating manual memory handling.

Garbage Collection

- **Automatic Memory Management:** Detects and removes unused objects to optimize memory usage, eliminating manual memory handling.
- **Reference Counting:** Primary mechanism to track object usage through a count of active references.

Garbage Collection

- **Automatic Memory Management:** Detects and removes unused objects to optimize memory usage, eliminating manual memory handling.
- **Reference Counting:** Primary mechanism to track object usage through a count of active references.
- **Cyclic Garbage Collector:** Handles circular references effectively, preventing memory leaks from interdependent objects.

Garbage Collection

- **Automatic Memory Management:** Detects and removes unused objects to optimize memory usage, eliminating manual memory handling.
- **Reference Counting:** Primary mechanism to track object usage through a count of active references.
- **Cyclic Garbage Collector:** Handles circular references effectively, preventing memory leaks from interdependent objects.
- **Three Generations:**
 - **Gen 0:** Newly created objects, collected most frequently
 - **Gen 1:** Objects surviving one collection cycle, collected less often
 - **Gen 2:** Long-lived objects, collected least frequently

Garbage Collection

1. Generation 0 (Young)

- Contains newly created objects
- Most frequently collected (every 700 objects)
- Quick collection cycle
- High turnover rate

Generation 0 (Young): In function `calculate_totals()`, temporary variables like `temp_sum` and loop calculations are created and quickly discarded. These objects are short-lived and immediately collected after use. Think of a calculator that clears its memory after each calculation, efficiently managing temporary values.

Garbage Collection

1. Generation 0 (Young)

- Contains newly created objects
- Most frequently collected (every 700 objects)
- Quick collection cycle
- High turnover rate

2. Generation 1 (Middle-Aged)

- Objects that survive Generation 0
- Collected after 10 Gen 0 collections
- Medium collection frequency
- More stable objects

Generation 0 (Young): In function `calculate_totals()`, temporary variables like `temp_sum` and loop calculations are created and quickly discarded. These objects are short-lived and immediately collected after use. Think of a calculator that clears its memory after each calculation, efficiently managing temporary values.

Generation 1 (Middle-Aged): A shopping cart example: `cart_items` list survives multiple operations as items are added and removed. It persists longer than temporary variables but isn't permanent. Like a shopping basket that stays active during your shopping session but gets cleaned up after checkout.

Garbage Collection

1. Generation 0 (Young)

- Contains newly created objects
- Most frequently collected (every 700 objects)
- Quick collection cycle
- High turnover rate

2. Generation 1 (Middle-Aged)

- Objects that survive Generation 0
- Collected after 10 Gen 0 collections
- Medium collection frequency
- More stable objects

3. Generation 2 (Old)

- Long-lived objects that survive Gen 1
- Least frequently collected
- Collected after 10 Gen 1 collections
- Most stable objects

Generation 0 (Young): In function `calculate_totals()`, temporary variables like `temp_sum` and loop calculations are created and quickly discarded. These objects are short-lived and immediately collected after use. Think of a calculator that clears its memory after each calculation, efficiently managing temporary values.

Generation 1 (Middle-Aged): A shopping cart example: `cart_items` list survives multiple operations as items are added and removed. It persists longer than temporary variables but isn't permanent. Like a shopping basket that stays active during your shopping session but gets cleaned up after checkout.

Generation 2 (Old): Program settings stored in `SETTINGS` dictionary live throughout the entire program's life. Created once at startup with essential configurations like API URLs and timeouts. These objects rarely change and stay in memory until the program ends, similar to a restaurant's business hours - set once, used throughout.

Garbage Collection: Advantages and Disadvantages

Advantages

- Automatic memory handling
- No manual allocation/deallocation needed
- Prevents memory leaks
- Efficient memory utilization
- Developer-friendly
- Safer memory management
- Can be manually controlled (gc module)

Garbage Collection: Advantages and Disadvantages

Advantages

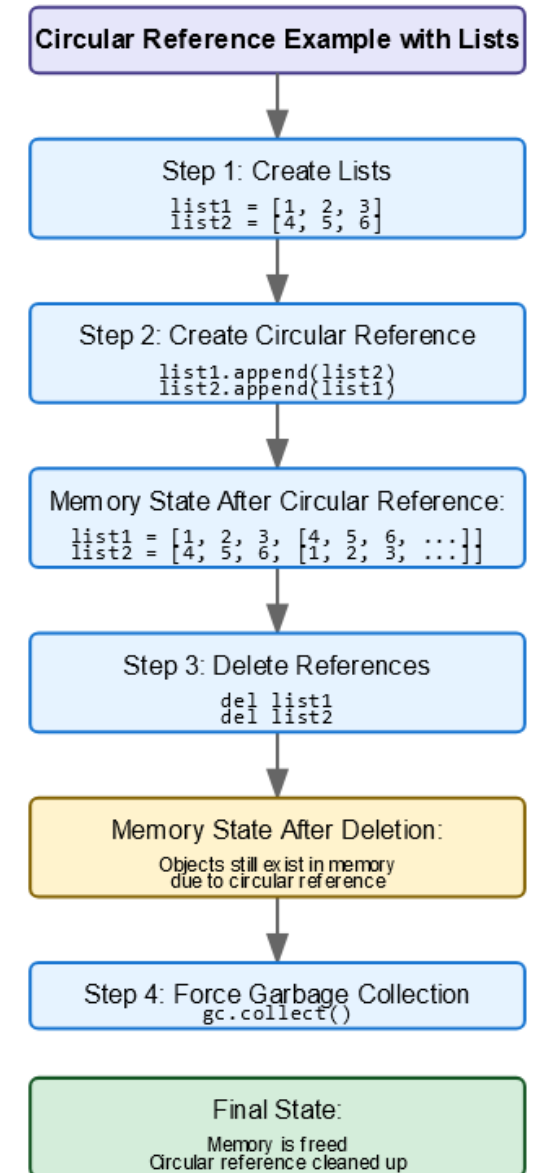
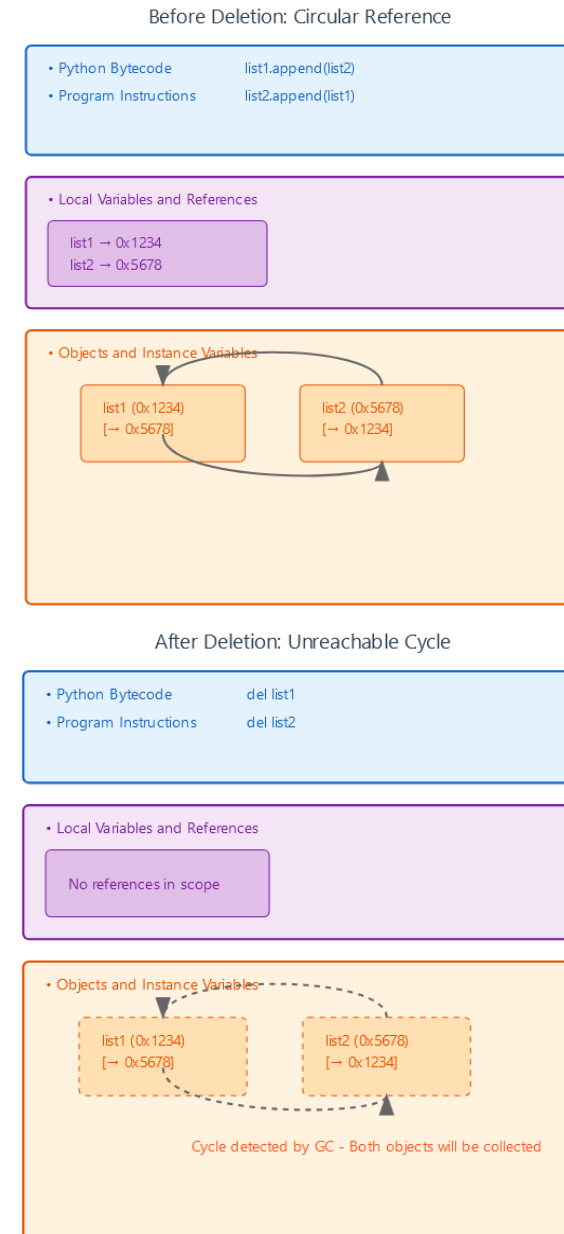
- Automatic memory handling
- No manual allocation/deallocation needed
- Prevents memory leaks
- Efficient memory utilization
- Developer-friendly
- Safer memory management
- Can be manually controlled (gc module)

Disadvantages

- Less control over memory
- Performance overhead
- Memory usage can be higher
- Timing of cleanup not predictable
- Resource intensive for large applications

Garbage Collection: Force garbage collection

```
1 # Import garbage collector module
2 import gc
3
4 list1 = [1, 2, 3]
5 list2 = [4, 5, 6]
6
7 # Create circular reference
8 list1.append(list2) # list1 references list2
9 list2.append(list1) # list2 references list1
10
11 # Remove references
12 del list1 # Reference count doesn't reach 0
13 del list2 # Due to circular reference #
14
15 Force garbage collection
16 gc.collect() # Cleans up circular reference
```



3. Memory Pooling

3. Memory Pooling

- An optimization technique used to manage memory allocation for small, frequently used objects.
- Pre-allocation of memory blocks for efficient reuse.
- Focuses on small objects like integers, strings, and frequently used types.

Memory Pooling: Key Characteristics

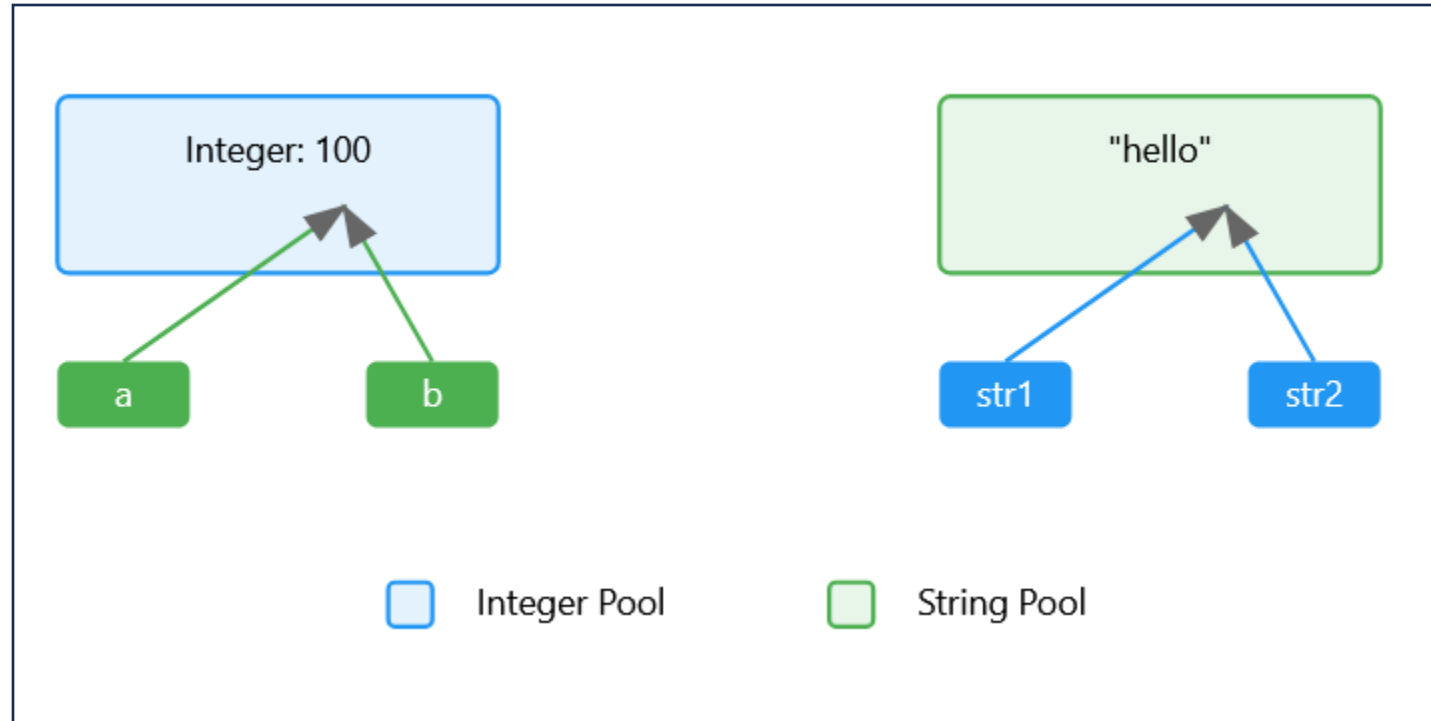
1. **Pre-allocation:** Memory blocks are reserved ahead of time to minimize the cost of repeated allocation.
2. **Fixed-size Pools:** Memory blocks are allocated in pools tailored to object sizes.
3. **Efficient Reuse:** Reuses blocks of memory instead of reallocating and deallocating memory repeatedly.
4. **Automatic Management:** Python handles memory pooling internally, no developer intervention needed.

Memory Pooling: Key Benefits

- **Reduces Overhead:** Pre-allocating memory reduces the need for frequent memory allocation and deallocation.
- **Minimizes Fragmentation:** Reuses memory blocks efficiently, reducing gaps in memory.
- **Improves Performance:** Faster memory allocation due to pre-allocation and reuse.
- **Optimized for Small Objects:** Especially beneficial for small integers, strings, and frequently used types.

Memory Pooling: Example

```
1 # Memory Pooling Examples
2 # Integers (small numbers)
3 a = 100
4 b = 100
5 print("Same integers:")
6 print(a is b) # True
7
8 # Short strings
9 str1 = "hello"
10 str2 = "hello"
11 print("\nShort strings:")
12 print(str1 is str2) # True
```



4. Memory Interning

Memory Interning

- Python uses an optimization technique called **interning** to save memory and speed up execution.
- Immutable objects like **strings** and **integers** are cached for reusability.
- Interned objects with the same value share the same memory address.
- Reduces memory overhead by avoiding duplication of immutable objects.
- Enhances performance when comparing frequently used objects.

Memory Interning: : Advantages & Disadvantages

Advantages:

- **Memory Efficiency:** Saves memory by storing a single copy of immutable objects.
- **Performance Boost:** Faster comparison using object identity (`is`) instead of value equality (`==`).

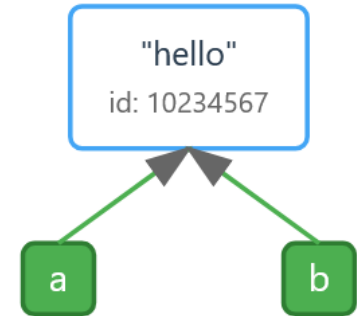
Disadvantages:

- **Limited Scope:** Works only for certain cases (e.g., small integers, short strings).
- **Manual Control Required:** Larger strings or non-standard cases need explicit interning.

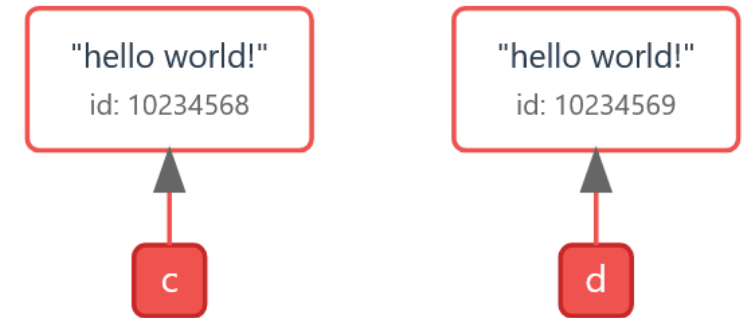
Memory Interning: : Example

```
1  # String literals share memory location
2  a = "hello"
3  b = "hello"
4
5  print(id(a)) # Memory location for 'a' (e.g., 140712834927872)
6  print(id(b)) # Memory location for 'b' (same as 'a')
7  print(a is b) # True - same memory location
8
9  # Longer strings - separate memory locations
10 c = "hello world!"
11 d = "hello world!"
12
13 print(id(c)) # Memory location for 'c' (e.g., 140712834928192)
14 print(id(d)) # Memory location for 'd' (different from 'c')
15 print(c is d) # False - different memory locations
```

String Literals Share Memory



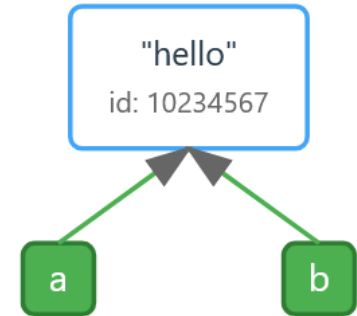
Longer Strings - Separate Memory



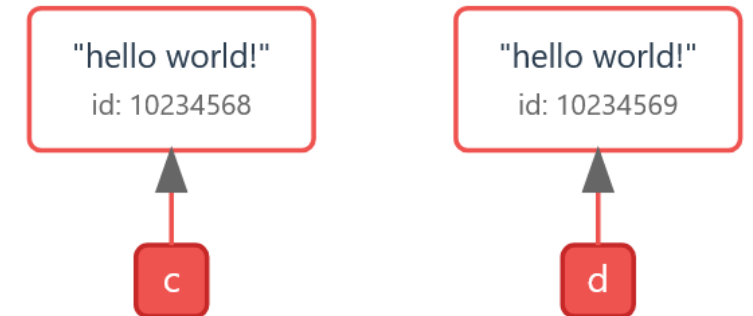
Memory Interning: : Example

```
1  # String literals share memory location
2  a = "hello"
3  b = "hello"
4
5  print(id(a)) # Memory location for 'a' (e.g., 140712834927872)
6  print(id(b)) # Memory location for 'b' (same as 'a')
7  print(a is b) # True - same memory location
8
9  # Longer strings - separate memory locations
10 c = "hello world!"
11 d = "hello world!"
12
13 print(id(c)) # Memory location for 'c' (e.g., 140712834928192)
14 print(id(d)) # Memory location for 'd' (different from 'c')
15 print(c is d) # False - different memory locations
```

String Literals Share Memory

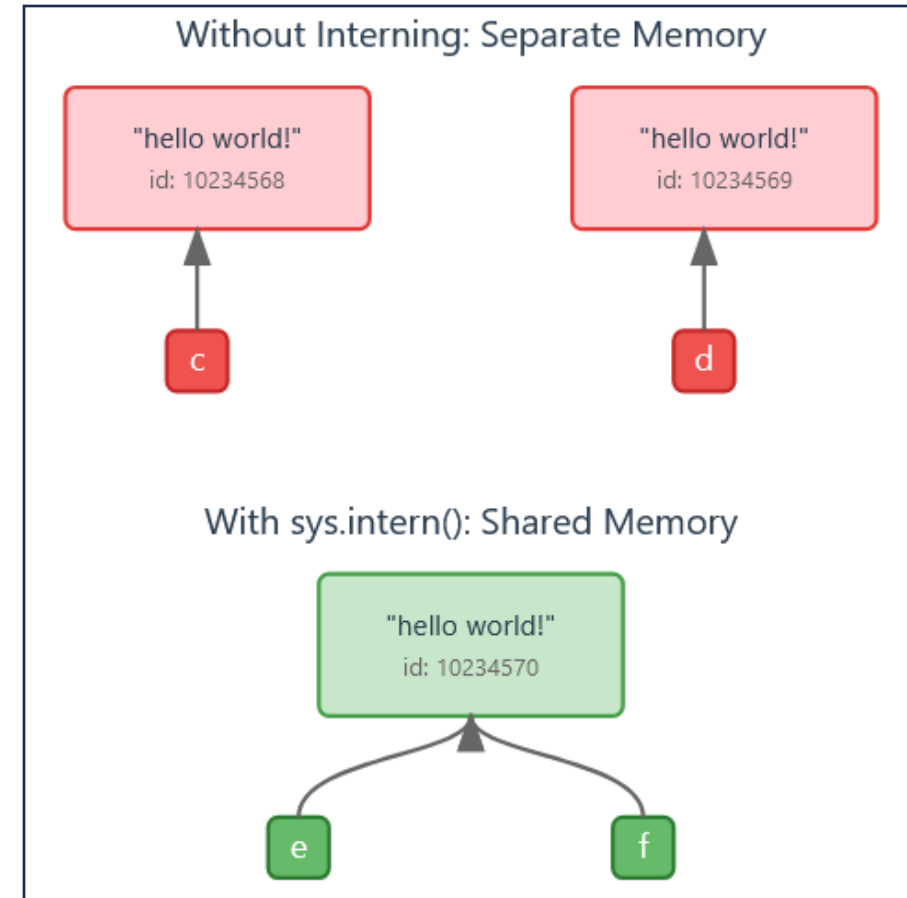


Longer Strings - Separate Memory



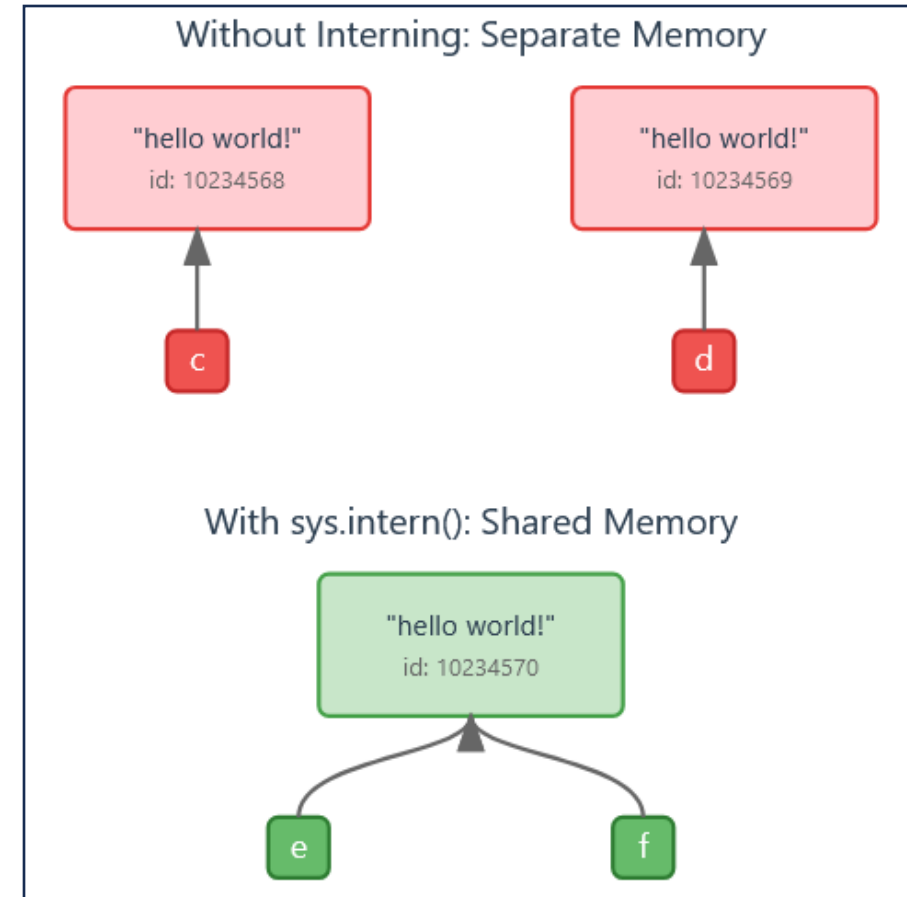
Memory Interning: : Manual Interning Example

```
1 import sys
2 # Longer strings - separate memory locations
3 c = "hello world!"
4 d = "hello world!"
5
6 print(id(c)) # Memory location for 'c' (e.g., 140712834928192)
7 print(id(d)) # Memory location for 'd' (different from 'c')
8 print(c is d) # False - different memory locations
9
10 # Manual interning for large strings
11 e = sys.intern("hello world!")
12 f = sys.intern("hello world!")
13 print(id(e)) # Memory location for 'e'
14 print(id(f)) # Memory location for 'f'
15 print(e is f) # True - same memory location due to manual interning
```



Memory Interning: : Manual Interning Example

```
1  import sys
2  # Longer strings - separate memory locations
3  c = "hello world!"
4  d = "hello world!"
5
6  print(id(c)) # Memory location for 'c' (e.g., 140712834928192)
7  print(id(d)) # Memory location for 'd' (different from 'c')
8  print(c is d) # False - different memory locations
9
10 # Manual interning for large strings
11 e = sys.intern("hello world!")
12 f = sys.intern("hello world!")
13 print(id(e)) # Memory location for 'e'
14 print(id(f)) # Memory location for 'f'
15 print(e is f) # True - same memory location due to manual interning
```



5. Manual Memory Management

Manual Memory Management

Overview

- Python uses automatic memory management via a garbage collector.
- Manual memory management is rarely required but can improve performance and control.

Manual Memory Management

Overview

- Python uses automatic memory management via a garbage collector.
- Manual memory management is rarely required but can improve performance and control.

When Manual Memory Management is Needed

- **Optimizing Memory Usage:** For applications with large datasets or memory-heavy operations.
- **Controlling Object Lifecycle:** In resource-constrained systems requiring precise control. [creation (allocating memory efficiently), usage (optimizing resource consumption), and release (ensuring timely deallocation of memory to avoid resource exhaustion)].

Manual Memory Management

Overview

- Python uses automatic memory management via a garbage collector.
- Manual memory management is rarely required but can improve performance and control.

When Manual Memory Management is Needed

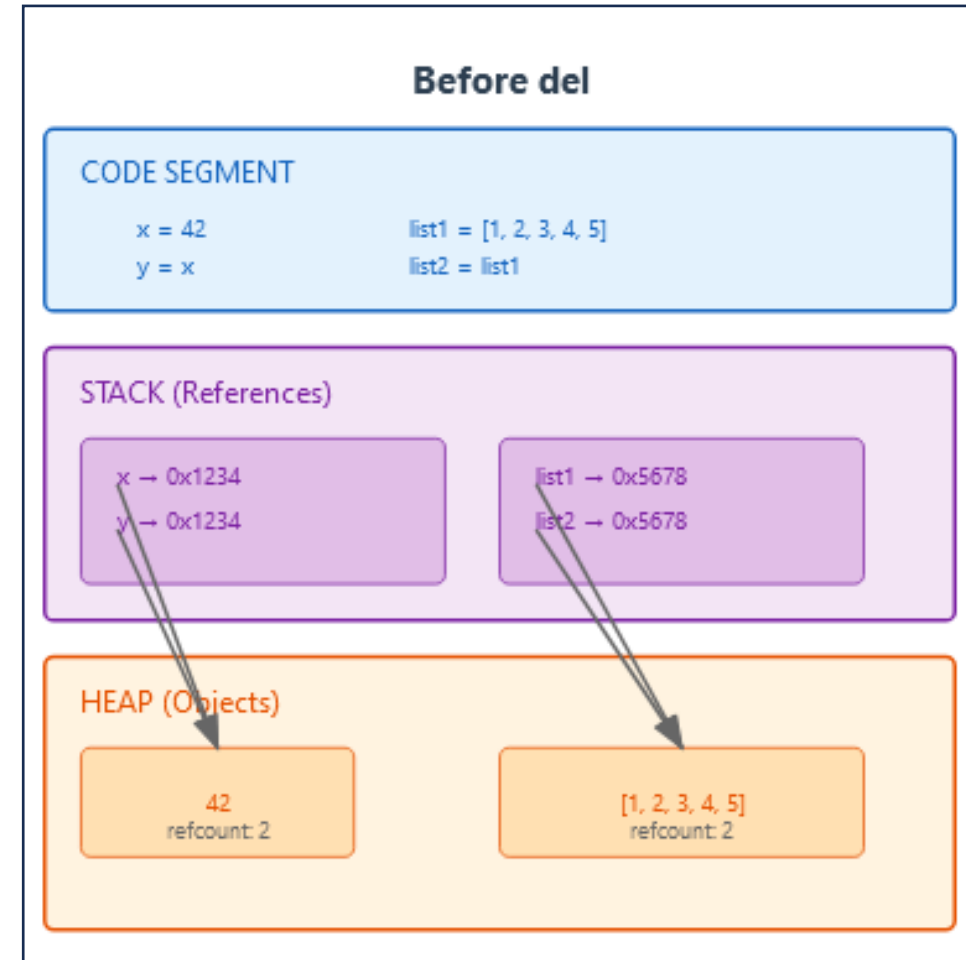
- **Optimizing Memory Usage:** For applications with large datasets or memory-heavy operations.
- **Controlling Object Lifecycle:** In resource-constrained systems requiring precise control. [creation (allocating memory efficiently), usage (optimizing resource consumption), and release (ensuring timely deallocation of memory to avoid resource exhaustion)].

Key Techniques

- **del Keyword:** Explicitly removes object references to free memory.
- **Weak References:** Use the weakref module to manage references without increasing the reference count.
- **Garbage Collection (gc):** Disable, enable, or manually trigger garbage collection.

Using del for Manual Object Deletion

```
1  # Create variables
2  x = 42
3  y = x
4  list1 = [1, 2, 3, 4, 5]
5  list2 = list1
6
7  # Delete all references
8  del x, y # Integer references deleted
9  del list1, list2 # List references deleted
10 # All objects now eligible for garbage collection
```



The **del** keyword allows developers to free up memory immediately by explicitly removing object references, improving performance, preventing memory leaks, and optimizing resource management in memory-intensive applications.

Using del for Manual Object Deletion

```
1  # Create variables
2  x = 42
3  y = x
4  list1 = [1, 2, 3, 4, 5]
5  list2 = list1
6
7  # Delete all references
8  del x, y # Integer references deleted
9  del list1, list2 # List references deleted
10 # All objects now eligible for garbage collection
```

After del x, y, list1, list2

CODE SEGMENT

```
del x, y
del list1, list2
```

STACK (References)

No references remain

HEAP (Objects)

Objects eligible for garbage collection

- No remaining references (refcount: 0)
- Memory will be freed by garbage collector

The **del** keyword allows developers to free up memory immediately by explicitly removing object references, improving performance, preventing memory leaks, and optimizing resource management in memory-intensive applications.

Using Weak References

```
1  import weakref
2  import sys # To check the reference count
3
4  def my_function():
5      return "Hello"
6
7  # Check the reference count before creating a weak reference
8  print(f"Reference count Before: {sys.getrefcount(my_function) - 1}")
9
10 # Create a weak reference to the function
11 weak_ref = weakref.ref(my_function)
12
13 # Check the reference count after creating a weak reference
14 print(f"Reference count after: {sys.getrefcount(my_function) - 1}")
15
16 # Access the function through the weak reference and call it
17 print(f"Accessing function through weak reference: {weak_ref() ()}") # Prints: Hello
18
19 # Delete the original reference to the function
20 del my_function
21
22 # Check the weak reference after the original function is deleted
23 print(f"Weak reference after deletion: {weak_ref()}") # Prints: None
```

Before Deletion

CODE SEGMENT

```
def my_function():
    return "Hello"
weak_ref = weakref.ref(my_function)
```

STACK

my_function → 0x1234
weak_ref → 0x1234 (weak)

HEAP

Function Object (my_function)
refcount: 1

Using Weak References

```
1  import weakref
2  import sys # To check the reference count
3
4  def my_function():
5      return "Hello"
6
7  # Check the reference count before creating a weak reference
8  print(f"Reference count Before: {sys.getrefcount(my_function) - 1}")
9
10 # Create a weak reference to the function
11 weak_ref = weakref.ref(my_function)
12
13 # Check the reference count after creating a weak reference
14 print(f"Reference count after: {sys.getrefcount(my_function) - 1}")
15
16 # Access the function through the weak reference and call it
17 print(f"Accessing function through weak reference: {weak_ref() ()}") # Prints: Hello
18
19 # Delete the original reference to the function
20 del my_function
21
22 # Check the weak reference after the original function is deleted
23 print(f"Weak reference after deletion: {weak_ref()}") # Prints: None
```

After deletion

CODE SEGMENT

```
del my_function
print(weak_ref()) # None
```

STACK

my_function → X (deleted)
weak_ref → None

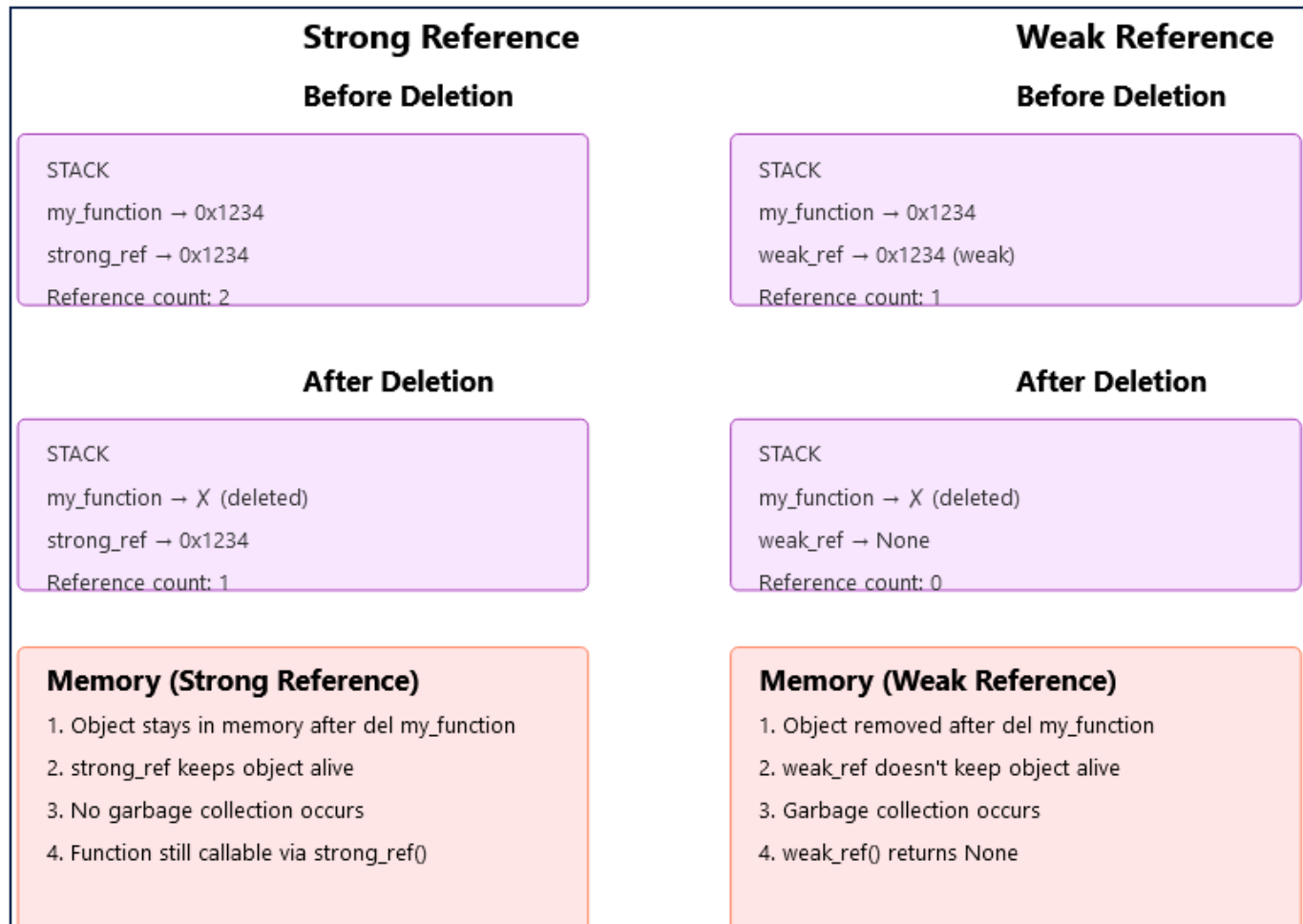
HEAP

Function Object (Garbage Collected)
refcount: 0

Using Weak References

Key Differences:

- Normal references increase reference count (Left refcount = 2).
- Weak references don't increase reference count (Right refcount = 1).
- Normal references prevent garbage collection, weak references don't.
- Normal references keep object alive, weak references allow collection when no strong references remain.



Using Garbage Collection (gc)

Manual garbage collection gives developers direct control over Python's garbage collector, allowing them to manage memory cleanup explicitly instead of relying on Python's automatic process.

Key Functions in gc

1. `gc.disable()`

- Turns off automatic garbage collection.
- Prevents interruptions during memory-intensive or time-sensitive operations.

1. `gc.collect()`

- Triggers garbage collection manually.
- Immediately frees up unused memory to avoid memory overflow or improve performance.

2. `gc.enable()`

- Re-enables automatic garbage collection.
- Restores Python's default memory management when manual control is no longer needed.

Using Garbage Collection (gc)

```
1  # Import garbage collector module
2  import gc
3
4  # Disable automatic garbage collection
5  gc.disable()
6  print("Automatic garbage collection disabled.")
7
8  list1 = [1, 2, 3]
9  list2 = [4, 5, 6]
10 list1.append(list2) # list1 references list2
11 list2.append(list1) # list2 references list1
12
13 # Remove references
14 del list1 # Reference count doesn't reach 0
15 del list2 # Due to circular reference #
16
17 # Manually trigger garbage collection
18 gc.collect() # Cleans up circular references
19 print("Garbage collection manually triggered and circular references cleaned up.")
20
21 # Re-enable automatic garbage collection
22 gc.enable()
23 print(" Automatic garbage collection re-enabled.")
```

CODE SEGMENT

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list1.append(list2) # Circular reference
list2.append(list1) # Circular reference
```

STACK (References)

list1 → 0x1234

list2 → 0x5678

HEAP (Objects)

0x1234:
[1, 2, 3, →]

refcount: 2

0x5678:
[4, 5, 6, →]

refcount: 2

Using Garbage Collection (gc)

```
1  # Import garbage collector module
2  import gc
3
4  # Disable automatic garbage collection
5  gc.disable()
6  print("Automatic garbage collection disabled.")
7
8  list1 = [1, 2, 3]
9  list2 = [4, 5, 6]
10 list1.append(list2) # list1 references list2
11 list2.append(list1) # list2 references list1
12
13 # Remove references
14 del list1 # Reference count doesn't reach 0
15 del list2 # Due to circular reference #
16
17 # Manually trigger garbage collection
18 gc.collect() # Cleans up circular references
19 print("Garbage collection manually triggered and circular references cleaned up.")
20
21 # Re-enable automatic garbage collection
22 gc.enable()
23 print(" Automatic garbage collection re-enabled.")
```

CODE SEGMENT

```
del list1 # Remove first reference
del list2 # Remove second reference
```

STACK (References)

Empty (all references deleted)

HEAP (Objects)

Memory leak: Objects still exist due to circular references

0x1234:

[1, 2, 3, →]

refcount: 1

0x5678:

[4, 5, 6, →]

refcount: 1



Using Garbage Collection (gc)

```
1  # Import garbage collector module
2  import gc
3
4  # Disable automatic garbage collection
5  gc.disable()
6  print("Automatic garbage collection disabled.")
7
8  list1 = [1, 2, 3]
9  list2 = [4, 5, 6]
10 list1.append(list2) # list1 references list2
11 list2.append(list1) # list2 references list1
12
13 # Remove references
14 del list1 # Reference count doesn't reach 0
15 del list2 # Due to circular reference #
16
17 # Manually trigger garbage collection
18 gc.collect() # Cleans up circular references
19 print("Garbage collection manually triggered and circular references cleaned up.")
20
21 # Re-enable automatic garbage collection
22 gc.enable()
23 print(" Automatic garbage collection re-enabled.")
```

CODE SEGMENT

```
gc.collect()
# Circular references cleaned up
```

STACK (References)

Empty (no references)

HEAP (Objects)

Memory Freed:

0x1234: [1, 2, 3, →] (collected)

0x5678: [4, 5, 6, →] (collected)

Garbage collector detected and removed circular references

GC